

TorUsNet 7.0 FR

L'Architech (Gaberiel Breton-Breault)

1 Introduction générale

1.1 — Avant-propos : évolution naturelle vers le nom TorUsNet

Le protocole initialement nommé *TorUsNet* évolue dans cette version 7.0 vers une identité plus naturelle : **TorUsNet**. Ce changement reflète une transition importante dans la philosophie du projet.

Dans ses premières versions, TorUsNet était conçu comme un protocole directionnel indépendant, démontrant la faisabilité d'une architecture de communication basée sur un tore privé inviolable et un tore public opaque. Avec TorUsNet 7.0, le protocole franchit une étape décisive : il est désormais pensé pour **s'intégrer naturellement dans un navigateur Tor**, comme couche de transport directionnelle remplaçant les mécanismes classiques basés sur TCP et TLS.

Le nom TorUsNet exprime cette double nature :

- **Tor** : intégration fluide dans l'écosystème du navigateur Tor.
- **Us** : continuité avec la géométrie du tore (torus), cœur de la sécurité directionnelle.
- **Net** : fonction première du protocole, une couche réseau non linéaire et inviolable.

TorUsNet n'est pas un fork de Tor. C'est une **architecture directionnelle** conçue pour remplacer la couche de transport tout en conservant l'esprit du navigateur Tor : anonymat, confidentialité, résilience et liberté d'accès.

Cette version 7.0 marque le début d'une convergence : **TorUsNet devient un protocole pensé pour Tor, avant d'être étendu au reste du réseau.**

1.2 — Introduction des nouveaux types de clés directionnelles (transmises sur le réseau)

TorUsNet 7.0 introduit un nouveau type de clé directionnelle, conçu pour fonctionner **exclusivement avec le modèle où le parcours directionnel est transmis sur le réseau**. Ces clés sont envoyées **avant le parcours** et servent à initialiser correctement le tore privé avant la reconstruction.

Structure des nouvelles clés

Chaque clé directionnelle possède la structure suivante :

- **20 bits — Point de départ** Indique la case initiale dans le tore privé où commence l'interprétation du parcours.
- **16 bits — ID de la case** Identifie la case visée dans le tore privé pour cette clé.

Résumé de la clé

$$\text{Clé directionnelle} = 20 \text{ bits (départ)} + 16 \text{ bits (ID)} = 36 \text{ bits}$$

Ces clés sont compactes, rapides à transmettre, et suffisantes pour initialiser le tore privé sans exposer sa géométrie interne.

Rôle dans le protocole

Ces clés :

- préparent le tore privé à recevoir le parcours complet,
- définissent la zone directionnelle initiale,
- assurent la cohérence entre le tore privé et le parcours transmis,
- permettent une reconstruction plus précise,
- garantissent que le tore privé interprète correctement les déplacements.

Elles sont transmises **avant** le parcours, car elles servent de pré-configuration directionnelle.

1.3 — Transmission directe des parcours directionnels

Dans les versions précédentes, le protocole utilisait un **seed** pour générer le parcours directionnel. TorUsNet 7.0 introduit une alternative : la **transmission directe du parcours directionnel complet**.

Désormais :

- le parcours directionnel peut être envoyé directement sur le réseau,
- le tore privé l'utilise tel quel pour reconstruire le fichier,
- aucune information interne du tore n'est exposée,
- la sécurité reste entièrement dans la géométrie interne.

Le parcours transmis ne contient :

- aucune clé interne,
- aucune structure du tore privé,
- aucune géométrie directionnelle,
- aucune information exploitable.

Il s'agit uniquement d'un **vecteur de déplacement**, sans valeur cryptographique.

1.4 — Transmission du hash du fichier reconstruit

Pour garantir que le fichier reconstruit par le tore privé est :

- complet,
- intact,
- non corrompu,
- cohérent avec le parcours transmis,

TorUsNet 7.0 introduit la transmission du **hash du fichier reconstruit**.

Ce hash :

- est envoyé sur le réseau,
- ne révèle aucune information directionnelle,
- ne permet aucune dérivation du tore,
- ne compromet aucune clé,
- et sert uniquement à valider l'intégrité du fichier final.

Il devient une preuve externe que la reconstruction directionnelle s'est déroulée correctement, même en cas de :

- perte de clés,
- parcours partiellement reçu,
- reconstructions multiples,
- interruptions réseau.

1.5 — Coexistence des deux modèles : seed ou parcours transmis

TorUsNet 7.0 permet désormais de **remplacer complètement le seed** par un **parcours intégral transmis directement sur le réseau**. Cependant, cette transmission directe alourdit légèrement le trafic réseau.

Pour cette raison, TorUsNet 7.0 conserve **les deux modèles** :

Modèle A — Seed directionnel (compact, local)

- Le seed de 70 bits est transmis.
- Le tore privé génère le parcours localement.
- Le trafic réseau est minimal.
- Idéal pour les environnements à faible bande passante.

Modèle B — Parcours directionnel transmis (complet, externe)

- Le parcours complet est envoyé sur le réseau.
- Le seed n'est plus nécessaire.
- Le protocole devient plus simple à implémenter.
- Idéal pour l'intégration dans un navigateur Tor.

Choix du modèle

Le choix entre seed ou parcours transmis doit être fait **au moment de la programmation**, selon :

- les contraintes du système,
- la bande passante,
- la simplicité recherchée,
- le type d'application,
- les préférences du développeur.

Les deux modèles sont **officiellement supportés** et documentés dans TorUsNet 7.0.

Chapitre 1 — Introduction à l'Architecture Directionnelle (TorUsNet 7.0)

1.1. Vision générale

L'architecture directionnelle est une technologie de communication entièrement nouvelle. Elle ne repose pas sur des mathématiques, des dérivations cryptographiques ou des structures classiques, mais sur une mécanique interne vivante, impossible à dériver, impossible à explorer, impossible à falsifier.

TorUsNet 7.0 s'appuie sur quatre éléments fondamentaux :

1. **Le Tore Privé** — une structure interne de 1 000 000 cases, chacune contenant :

- 1 bit normal,
- 1 bit fantôme implicite (NOT du bit normal),
- 4 directions possibles. Le tore privé est unique à chaque session, jamais transmis, jamais exporté.

2. **La Mécanique Directionnelle** — un ensemble de règles définissant :

- comment une clé se déplace dans le tore,
- comment les IDs mutent,
- comment les chemins internes s'activent,
- comment les paires normal/fantôme s'inversent,
- comment la cohérence interne est maintenue.

3. **La Forteresse de Verre** — un module logiciel interne qui :

- valide la cohérence directionnelle,
- surveille les ports directionnels,
- surveille les IDs internes,
- détruit la session en cas d'anomalie,
- bloque les IP hostiles.

4. **Les Clés Directionnelles Transmises (TorUsNet 7.0)** — un nouveau type de clé envoyé sur le réseau avant le parcours :

- **20 bits : point de départ,**

- **16 bits : ID de la case**, soit **36 bits** par clé. Ces clés préparent le tore privé à recevoir le parcours transmis.

L'architecture directionnelle n'utilise :

- ni chiffrement,
- ni dérivation mathématique,
- ni signatures,
- ni certificats,
- ni clés privées,
- ni primitives cryptographiques.

Elle repose exclusivement sur :

- des fenêtres directionnelles,
- des constantes directionnelles,
- des parcours internes,
- des mutations d'ID,
- des chemins internes,
- des paires normal/fantôme,
- des structures internes non dérivables.

1.2. Pourquoi une architecture directionnelle ?

Les protocoles classiques (TLS, SSH, IPSec, WireGuard, etc.) reposent sur :

- des mathématiques,
- des dérivations,
- des clés privées,
- des certificats,
- des signatures.

Ces systèmes sont vulnérables à :

- l'analyse mathématique,

- les ordinateurs quantiques,
- les ordinateurs photoniques,
- les attaques par dérivation,
- les collisions,
- les attaques par répétition,
- les injections.

L'architecture directionnelle élimine entièrement ces vecteurs d'attaque. Elle ne repose sur aucune structure mathématique exploitable. Elle repose sur une mécanique interne impossible à dériver, impossible à explorer, impossible à inverser.

Même avec une puissance infinie, un attaquant ne peut pas :

- dériver un tore privé,
- explorer un tore privé,
- reconstruire un tore privé,
- prédire un parcours directionnel,
- falsifier une clé directionnelle,
- répéter une clé directionnelle,
- reconstruire un ID interne.

1.3. Les trois éléments fondamentaux (version 7.0)

1.3.1. Le Tore Privé

Le Tore Privé est une structure interne de 1 000 000 cases, chacune contenant :

- 1 bit normal,
- 1 bit fantôme implicite,
- 4 directions possibles.

Il est :

- unique à chaque session,
- reconstruit fractalement,
- jamais transmis,

- jamais exporté,
- jamais dérivable.

Il contient :

- les IDs internes,
- l'ID global,
- l'ID unique du tore,
- les chemins internes,
- les paires normal/fantôme.

Inversion des bits normal/fantôme lors du parcours

Dans TorUsNet 7.0, **chaque fois qu'une case est traversée par le parcours directionnel**, les bits normal et fantôme **s'inversent automatiquement** :

- normal = 0 $\rightarrow$ 1
- fantôme = 1 $\rightarrow$ 0

ou inversement.

Cette inversion :

- crée une oscillation directionnelle,
- modifie la géométrie interne du tore,
- empêche toute prédiction,
- renforce la non-dérivabilité,
- rend chaque parcours unique et non reproductible.

Cette règle est fondamentale dans TorUsNet 7.0.

1.3.2. La Mécanique Directionnelle

Elle définit :

- comment une clé se déplace dans le tore,
- comment les IDs mutent,
- comment les chemins internes s'activent,

- comment les paires normal/fantôme s'inversent,
- comment la cohérence interne est maintenue.

Elle est :

- non linéaire,
- non dérivable,
- non réversible,
- vivante,
- oscillante,
- mutante.

1.3.3. La Forteresse de Verre

La Forteresse de Verre :

- valide la cohérence directionnelle,
- surveille les ports directionnels,
- surveille les IDs internes,
- détruit la session en cas d'anomalie,
- bloque les IP hostiles.

Elle est l'équivalent directionnel d'un pare-feu structurel.

1.4. Ports directionnels : ports numériques étendus

Les ports directionnels sont :

- des ports numériques classiques,
- mais avec une plage étendue dans les milliards,
- générés par les clés directionnelles,
- validés par la Forteresse de Verre.

Ils servent à :

- transporter les transmissions directionnelles,
- éviter les collisions,

- empêcher les scans,
- empêcher les injections,
- empêcher les détournements de flux.

Exemples :

- 1 482 993 112
- 7 000 221 991
- 3 991 000 000

Impossible à scanner. Impossible à prédire. Impossible à falsifier.

1.5. Une architecture post-Internet

L'architecture directionnelle est conçue pour l'ère :

- post quantique,
- post cryptographique,
- photonique.

Elle ne dépend pas :

- des mathématiques,
- des certificats,
- des clés privées,
- des primitives cryptographiques.

Elle utilise encore :

- DNS,
- HTTPS,

mais uniquement comme sas d'entrée temporaire avant que la session ne devienne 100 % directionnelle.

1.6. Une session directionnelle : vivante et autonome

Une session directionnelle est :

- vivante,

- autonome,
- auto-cohérente,
- auto-validante,
- auto-mutante.

Elle évolue à chaque clé :

- mutation d’ID,
- inversion normal/fantôme,
- progression interne,
- mise à jour du tore privé,
- validation par la Forteresse de Verre.

Elle ne peut pas être :

- copiée,
- clonée,
- dérivée,
- prédite,
- falsifiée.

1.7. Objectifs du protocole directionnel

TorUsNet vise à :

1. Créer des sessions inviolables par conception.
2. Remplacer les protocoles cryptographiques classiques.
3. Créer des réseaux privés directionnels (multi-tores).
4. Permettre la reconstruction directionnelle de fichiers.
5. Créer une couche de communication interne (ports directionnels).
6. Créer une identité directionnelle unique.
7. Assurer une sécurité post-quantique et photonique.

1.8. Résumé du Chapitre 1

L'architecture directionnelle :

- n'est pas cryptographique,
- n'est pas mathématique,
- n'est pas dérivable,
- n'est pas falsifiable.

Elle repose sur :

- un tore privé,
- une mécanique interne,
- une cohérence directionnelle,
- des ports numériques étendus,
- une Forteresse de Verre logicielle,
- des clés directionnelles transmises,
- un parcours transmis ou généré par seed,
- un hash final validant la reconstruction,
- **l'inversion automatique des bits normal/fantôme lors du parcours.**

Elle constitue une nouvelle génération de protocole, conçue pour l'ère post-quantique et photonique.

Chapitre 2 — Refonte du Seed Directionnel et Introduction du Modèle à Parcours Transmis (TorUsNet 7.0)

TorUsNet 7.0 introduit une évolution majeure dans la manière dont les trajectoires directionnelles sont générées, transmises et interprétées. Deux modèles coexistent désormais :

1. **Le modèle classique basé sur le seed directionnel (70 bits)**
2. **Le modèle moderne basé sur le parcours transmis directement sur le réseau**

Les deux modèles sont officiellement supportés et documentés.

2.1 — Structure de la clé directionnelle (modèle seed, 128 bits)

La clé directionnelle complète conserve sa structure originale de **128 bits**, composée de :

- **2 bits — Type de clé**
- **20 bits — Point de départ directionnel**
- **20 bits — Point d'arrivée directionnel**
- **16 bits — ID de case visée**
- **70 bits — Seed directionnel**

Total : 128 bits

2.2 — Structure du seed directionnel (70 bits)

Le seed directionnel est un bloc compact de 70 bits contenant cinq composantes :

1. **Base exponentielle (4 bits)**
2. **Sélection de la tour fixe (4 bits)**
3. **Hauteur de la tour fixe (2 bits)**
4. **Étages dynamiques (48 bits)**
5. **Modulation finale (12 bits)**
 - Addition : 4 bits
 - Multiplication : **8 bits**

2.2.1 — Base exponentielle (4 bits)

Les 4 bits définissent une base parmi 16 valeurs hiérarchiques :

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 100, 1000, 10 000, 100 000, 1 000 000, 10 000 000

2.2.2 — Sélection de la tour fixe (4 bits)

Liste complète des 15 tours fixes disponibles

Les tours fixes continuent leur logique d'expansion jusqu'à **15 possibilités**, chacune suivant la règle :

$$n^{(n+1)(n+2)(n+3)}$$

où $n \in [1, 15]$.

Voici la liste complète :

1. **Tour fixe 1** : $1^{2^3^4}$
2. **Tour fixe 2** : $2^{3^4^5}$
3. **Tour fixe 3** : $3^{4^5^6}$
4. **Tour fixe 4** : $4^{5^6^7}$
5. **Tour fixe 5** : $5^{6^7^8}$
6. **Tour fixe 6** : $6^{7^8^9}$
7. **Tour fixe 7** : $7^{8^9^{10}}$
8. **Tour fixe 8** : $8^{9^{10^{11}}}$
9. **Tour fixe 9** : $9^{10^{11^{12}}}$
10. **Tour fixe 10** : $10^{11^{12^{13}}}$
11. **Tour fixe 11** : $11^{12^{13^{14}}}$
12. **Tour fixe 12** : $12^{13^{14^{15}}}$
13. **Tour fixe 13** : $13^{14^{15^{16}}}$
14. **Tour fixe 14** : $14^{15^{16^{17}}}$
15. **Tour fixe 15** : $15^{16^{17^{18}}}$

Code 0000 : aucune tour fixe utilisée

Les 4 bits permettent donc :

- **0000** → aucune tour fixe
- **0001 à 1111** → sélection d'une tour fixe parmi les 15 possibilités

2.2.3 — Hauteur de la tour fixe (2 bits)

Détermine le nombre d'étages utilisés :

- 00 → 1 étage
- 01 → 2 étages

- 10 → 3 étages
- 11 → 4 étages

2.2.4 — Étages dynamiques (48 bits)

Les 48 bits sont divisés en **6 étages dynamiques**, chacun représenté par une valeur brute sur **8 bits** :

$$D_1, D_2, D_3, D_4, D_5, D_6 \in \{0, 1, \dots, 255\}$$

Règle directionnelle :

- si un étage dynamique vaut **0**, il est **ignoré**,
- si sa valeur est **> 0**, il est **actif** dans la tour exponentielle.

2.2.5 — Modulation finale (12 bits)

La modulation finale est composée de :

- **Addition (A)** — 4 bits → 0 à 15
- **Multiplication (M)** — 8 bits → 0 à 255

La valeur exponentielle finale devient :

$$E_{\text{final}} = (X + A) \times M$$

où **X** représente la tour complète :

$$X = \text{Base}^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}$$

2.3 — Structure exponentielle finale

Avec tour fixe :

$$\text{Base}^{e_1^{e_2^{e_3^{e_4^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}}}}}$$

Sans tour fixe :

$$\text{Base}^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}$$

Puis modulation :

$$E_{\text{final}} = (X + A) \times M$$

2.4 — Capacité de représentation

- **Espace brut :**

$$2^{48}$$

- **Espace exponentiel :** dépasse largement les capacités cryptographiques, quantiques et photoniques.

2.5 — Impact sur le tore privé

Le tore privé utilise :

- la base exponentielle,
- la tour fixe (si activée),
- les 6 étages dynamiques,
- la modulation finale (A et M),
- la règle d'ignorance des étages nuls,

pour déterminer la trajectoire directionnelle interne.

Inversion normal/fantôme

Lors de l'exécution du parcours :

- chaque case traversée inverse son bit normal et son bit fantôme,
- modifiant la géométrie interne du tore,
- rendant chaque parcours unique et non reproductible.

2.6 — Introduction du modèle à parcours transmis (TorUsNet 7.0)

TorUsNet 7.0 introduit un second modèle :

Le parcours directionnel complet est transmis directement sur le réseau.

Ce modèle :

- supprime l'utilisation du seed,
- simplifie la génération directionnelle,
- permet une intégration naturelle dans un navigateur Tor,
- augmente légèrement le trafic réseau,
- nécessite les nouvelles clés directionnelles (36 bits).

Ordre de transmission :

1. **Clés directionnelles (36 bits)**
2. **Parcours directionnel complet**
3. **Hash du fichier reconstruit**

2.7 — Conclusion du Chapitre 2

TorUsNet 7.0 introduit :

- un seed directionnel de 70 bits,
- un moteur exponentiel multi-étages,
- **15 tours fixes,**
- une hauteur configurable,
- **6 étages dynamiques sur 48 bits,**
- une modulation finale sur **12 bits** (4 bits addition + **8 bits multiplication**),
- un modèle basé sur le parcours transmis,
- de nouvelles clés directionnelles (36 bits),
- la transmission du hash final,
- l'inversion automatique des bits normal/fantôme.

Cette structure renforce la nature inviolable, non dérivable et géométriquement imprévisible de TorUsNet.

Chapitre 3 — Forteresse de Verre (Pare-feu directionnel structurel, TorUsNet 7.0)

La Forteresse de Verre est le pare-feu directionnel structurel du tore privé. Elle valide, surveille et protège toutes les opérations directionnelles, selon le modèle choisi par les programmeurs :

- **Modèle A : Clé directionnelle transmise (36 bits)**
- **Modèle B : Clé directionnelle complète (128 bits)**

Les deux modèles sont supportés, mais **ne doivent jamais être utilisés simultanément**. La Forteresse de Verre s'adapte automatiquement au modèle actif.

3.1. Vue d'ensemble

La Forteresse de Verre valide :

- la clé directionnelle (36 bits **ou** 128 bits selon le modèle),
- l'ID transmis (16 bits),
- l'ID reconstruit (16 bits),
- la cohérence fractale du tore,
- la reconstruction directionnelle du fichier,
- l'inversion normal/fantôme lors du parcours.

Elle protège :

- le tore privé,
- la clé directionnelle du modèle actif,
- la cohérence directionnelle,
- la reconstruction du fichier.

Elle détruit la session en cas d'anomalie.

3.2. Rôle fondamental

✓ Validation

✓ Surveillance

✓ Protection

✓ Destruction

3.3. Validation conjointe : Forteresse de Verre + Host

Peu importe le modèle choisi (36 bits ou 128 bits), la Forteresse de Verre **valide toujours la clé directionnelle avec le Host.**

✓ Fonctionnement

1. Le Host reçoit la clé directionnelle (36 bits **ou** 128 bits).
2. Le Host effectue une pré-validation directionnelle.
3. La Forteresse de Verre effectue la validation structurelle finale.

✓ Si les deux valident la clé :

La Forteresse de Verre autorise :

- la mutation des 16 bits internes,
- la reconstruction interne (selon le modèle),
- l'exécution du parcours,
- l'inversion normal/fantôme.

✗ Si l'un des deux rejette la clé :

La Forteresse de Verre :

- bloque toute mutation,
- bloque toute reconstruction,
- détruit la session,
- efface le tore privé,
- bloque l'IP.

✓ Règle directionnelle absolue

La clé directionnelle doit être validée **par le Host ET par la Forteresse de Verre.**

3.4. Modèle A — Validation de la clé directionnelle transmise (36 bits)

Ce modèle est utilisé lorsque les programmeurs choisissent de **transmettre la clé directionnelle sur le réseau.**

La Forteresse de Verre vérifie :

1. Les 20 bits du point de départ

- case existante,
- cohérence directionnelle,
- cohérence fractale.

2. Les 16 bits de l'ID transmis

- ID existant,
- ID non corrompu,
- ID non falsifié.

3. Les 16 bits de l'ID reconstruit

- cohérence interne,
- mutation valide,
- compatibilité avec le modèle 36 bits.

4. Les 36 bits de la clé directionnelle transmise

- cohérence fractale,
- absence de dérive,
- absence de falsification.

Si un seul bit est incohérent → **session détruite**.

3.5. Modèle B — Validation de la clé directionnelle complète (128 bits)

Ce modèle est utilisé lorsque les programmeurs choisissent de **ne pas transmettre la clé**, mais de la reconstruire entièrement dans le tore privé.

La Forteresse de Verre vérifie :

1. Les 20 bits du point de départ

2. Les 16 bits de l'ID transmis

3. Les 16 bits de l'ID reconstruit

4. Les 128 bits de la clé directionnelle complète

La clé directionnelle de 128 bits est :

- la seule clé qui mute,
- la seule clé qui inverse,
- la seule clé qui oscille,
- la seule clé qui se déplace.

Si la clé directionnelle est incohérente → **session détruite**.

3.6. Règle structurelle : les IDs de case peuvent se répéter

✓ **Un ID de case peut se répéter sans aucune limite.** Il n'existe plus de plafond de 4 cases.

✓ Un ID peut être partagé par : → 10 cases, → 100 cases, → 1 000 cases, → 50 000 cases, → ou davantage.

✓ Cela ne compromet pas la sécurité, car elle repose sur :

- la géométrie directionnelle,
- les mutations internes,
- les déplacements directionnels,
- la cohérence fractale,
- l'inversion normal/fantôme.

✓ **50 % des 16 bits sont réservés aux mutations directionnelles.** Les autres 50 % sont les IDs actifs, mais leur distribution est **libre et non contrainte**.

✓ Les 1 000 000 de cases du tore privé peuvent donc :

- partager massivement les mêmes IDs,
- se regrouper en clusters directionnels,
- muter vers des IDs déjà utilisés,
- se superposer fractalement.

Ce n'est pas une collision : → **c'est une propriété directionnelle normale, attendue et validée.**

✓ Rôle de la Forteresse de Verre

Elle valide que :

- les répétitions d'ID sont légales,
- les mutations vers des IDs déjà utilisés sont cohérentes,
- la fractale interne reste stable malgré les regroupements,
- aucune dérive directionnelle n'est détectée.

3.7. Validation de la reconstruction du fichier

La Forteresse de Verre vérifie :

- la cohérence de la clé directionnelle (36 bits ou 128 bits selon le modèle),
- la cohérence du tore,
- la cohérence fractale,
- l'absence de falsification,
- l'absence de dérive.

Elle vérifie aussi que :

- la taille du fichier n'est jamais transmise,
- la taille du fichier n'est jamais visible,
- la reconstruction est purement directionnelle.

Si une incohérence est détectée :

- la session est détruite,
- le tore privé est effacé,
- la clé directionnelle est détruite,
- la reconstruction est annulée,
- l'IP est bloquée.

3.8. Protection structurelle

La Forteresse de Verre protège :

✓ Le tore privé ✓ La clé directionnelle du modèle actif ✓ Le fichier ✓ La cohérence fractale
✓ La cohérence directionnelle

Elle rend toute falsification impossible par conception.

3.9. Destruction automatique

Si une incohérence est détectée :

- la session est détruite,
- le tore privé est effacé,
- la clé directionnelle est détruite,
- la reconstruction est annulée,
- l'IP est bloquée.

La destruction est :

- **immédiate,**
- **irréversible,**
- **totale.**

3.10. Résumé du Chapitre 3

La Forteresse de Verre :

- valide la clé directionnelle **selon le modèle choisi** (36 bits ou 128 bits),
- valide l'ID transmis (16 bits),
- valide l'ID reconstruit (16 bits),
- valide la clé conjointement avec le Host,
- valide la reconstruction du fichier,
- accepte les IDs répétés sans limite,
- protège le tore privé,
- détruit la session en cas d'anomalie.

Elle est le **pare-feu directionnel structurel**, inviolable par conception.

Chapitre 4 — Tore Privé (Structure interne, géométrie, fichiers, directions, IDs)

(version mise à jour avec la correction sur la clé 36 bits)

4.1. Vue d'ensemble

Le Tore Privé est la structure directionnelle interne utilisée pour :

- parcourir les directions internes,
- exécuter les sauts exponentiels,
- muter les IDs,
- inverser normal/fantôme,
- reconstruire le fichier,
- stabiliser la cohérence fractale.

Il contient :

- **1 000 000 cases,**
- chacune avec :
 - un **ID sur 16 bits,**
 - un **bit normal,**
 - un **bit fantôme implicite,**
 - **4 directions internes,**
 - un **déplacement torique,**
 - un **saut exponentiel circulaire.**

Le Tore Privé est programmé avec **deux fichiers distincts**, lus en parallèle.

4.2. Structure interne du tore

Chaque case du tore contient :

✓ 1. Un ID sur 16 bits

- 100 % des valeurs possibles (0 à 65535) sont utilisées.
- Les IDs peuvent se répéter **sans aucune limite.**
- Aucune contrainte d'unicité, de collision ou de disponibilité.

✓ 2. Un bit normal

- 0 = normal
- 1 = fantôme (implicite, jamais transmis)

✓ 3. Quatre directions internes

Codées sur 2 bits :

Code Direction

- 00** direction 1
- 01** direction 2
- 10** direction 3
- 11** direction 4

✓ 4. Cohérence fractale

Chaque case possède une cohérence interne qui :

- se met à jour à chaque mutation,
- se met à jour à chaque inversion normal/fantôme,
- se met à jour à chaque déplacement directionnel.

4.3. Déplacements cardinaux et diagonaux (comportement torique)

Les déplacements internes se comportent comme un tore parfait :

✓ Dépasser en haut → arrivée en bas ✓ Dépasser en bas → arrivée en haut ✓ Dépasser à gauche → arrivée à droite ✓ Dépasser à droite → arrivée à gauche

Ce comportement :

- supprime les bords,
- supprime les limites,
- supprime les zones mortes,
- garantit une continuité parfaite,
- rend impossible toute sortie du tore.

✓ Les déplacements diagonaux suivent la même logique.

4.4. Sauts exponentiels (comportement circulaire)

Chaque direction interne contient un **saut exponentiel**.

Ce saut :

- ne suit pas les axes cardinaux,
- ne suit pas les diagonales,
- ne suit pas la grille,
- ne suit pas la géométrie du tore.

Il suit une **orbite circulaire interne**.

✓ Le saut exponentiel se déplace sur un cercle. ✓ Lorsqu'il atteint la fin du cercle, il revient au début. ✓ Il ne peut jamais sortir de l'orbite. ✓ Il ne peut jamais se bloquer. ✓ Il ne peut jamais dériver.

Ce comportement circulaire :

- amplifie la portée directionnelle,
- crée une géométrie interne indépendante,
- rend les parcours impossibles à inverser,
- stabilise la cohérence fractale,
- empêche toute dérivation externe.

4.5. Géométrie hybride : tore + cercle

La structure interne combine deux géométries :

✓ Géométrie torique

Pour les déplacements cardinaux et diagonaux.

✓ Géométrie circulaire

Pour les sauts exponentiels.

Cette dualité :

- crée une surface directionnelle vivante,
- modifie la géométrie selon le type de déplacement,
- rend les parcours impossibles à inverser,
- stabilise la cohérence fractale,

- empêche toute dérivation externe.

4.6. Architecture logicielle du Tore Privé

Le Tore Privé est programmé avec **deux fichiers distincts**, chacun ayant un rôle précis.

4.6.1. Fichier des bits normaux (binaire pur, chargé en RAM)

Ce fichier :

- contient uniquement les bits normaux,
- est un fichier binaire pur,
- est chargé entièrement en RAM,
- ne contient aucune direction,
- ne contient aucun ID,
- ne contient aucun saut exponentiel.

Le chargement en RAM permet :

- une mutation instantanée des cases,
- une lecture ultra rapide des bits normaux,
- une reconstruction continue du fichier,
- une inversion normal/fantôme immédiate.

4.6.2. Fichier directionnel (texte, reste sur le disque dur)

Ce fichier :

- reste sur le disque dur,
- est un fichier texte,
- contient les 4 directions de chaque case,
- contient les sauts exponentiels,
- contient les IDs de chaque case,
- contient **1 000 000 lignes**,
- est parcouru en parallèle avec le fichier binaire.

Structure d'une ligne :

Code

<ID> 101 2^5, 110 3^7, 101 2^7, 111 4^7

4.7. Mutation des IDs

Deux types de mutation existent, selon le modèle directionnel choisi.

4.7.1. Mutation locale (modèle 36 bits)

(corrigé selon la règle demandée)

Dans le modèle 36 bits :

- la clé transmise fournit uniquement :
 - la case de départ,
 - l'ID de cette case ;
- la clé ne parcourt pas le tore ;
- la clé n'exécute aucun déplacement ;
- la mutation locale est effectuée **par l'exécution du parcours directionnel interne**,
- la clé ne mute jamais directement,
- aucune inversion normal/fantôme n'est effectuée,
- aucune autre case n'est modifiée.

Mutation locale :

$$ID_{\text{case}} \leftarrow ID_{\text{resultat du parcours}}$$

La mutation est donc **une conséquence du déplacement directionnel**, et non une action directe de la clé.

4.7.2. Mutation structurelle (modèle 128 bits)

Dans le modèle 128 bits :

- la clé interne parcourt le tore,
- inverse normal/fantôme,
- reconstruit le fichier,
- mute l'ID global du tore,

- mute l'ID de la case de départ,
- modifie la cohérence fractale.

Mutation structurelle :

$$ID_{\text{tore}} \leftarrow ID_{\text{aleatoire}}$$

$$ID_{\text{case départ}} \leftarrow ID_{\text{aleatoire}}$$

4.8. Normal / Fantôme

Chaque case possède :

- un bit normal,
- un bit fantôme implicite.

✓ Le modèle 128 bits peut inverser normal/fantôme. ✗ Le modèle 36 bits ne peut pas inverser normal/fantôme.

4.9. ID du tore

Le tore possède un **ID global sur 16 bits**.

Cet ID :

- change à chaque clé interne (modèle 128 bits),
- est choisi aléatoirement sur 16 bits,
- peut se répéter librement,
- n'a aucune contrainte d'unicité.

4.10. Rôle du tore dans la reconstruction du fichier

Le Tore Privé :

- reconstruit le fichier bit par bit,
- stabilise la cohérence fractale,
- applique les mutations directionnelles,
- applique les inversions normal/fantôme,
- applique les sauts exponentiels,
- applique les déplacements toriques.

La taille du fichier :

- n'est jamais transmise,
- n'est jamais visible,
- est déduite directionnellement.

4.11. Résumé du Chapitre 4

Le Tore Privé :

- contient 1 000 000 cases,
- chaque case possède un ID sur 16 bits,
- les IDs peuvent se répéter librement et massivement,
- les déplacements cardinaux sont toriques,
- les sauts exponentiels sont circulaires,
- la géométrie hybride rend les parcours impossibles à inverser,
- le tore est programmé avec deux fichiers (binaire RAM + directionnel disque),
- le modèle 36 bits fournit la case de départ et son ID,
- la mutation locale du modèle 36 bits est effectuée par le parcours directionnel interne,
- le modèle 128 bits mute structurellement,
- l'inversion normal/fantôme dépend du modèle choisi,
- la cohérence fractale est mise à jour en continu,
- la structure hybride rend le tore inviolable.

Chapitre 5 — Système des ID (16 bits, répétition, mutation, rôle)

(Version TorUsNet 7.0 — mise à jour complète)

5.1. Vue d'ensemble

Le protocole directionnel utilise un système d'identifiants (ID) entièrement basé sur :

- des IDs sur **16 bits**,

- **100 % des valeurs possibles,**
- des IDs **répétables sans limite,**
- des mutations **aléatoires,**
- des redéfinitions **locales ou structurelles,**
- une interaction dépendante du modèle directionnel choisi :
 - **modèle 36 bits** (mutation locale via parcours),
 - **modèle 128 bits** (mutation structurelle).

Les IDs sont au cœur :

- du Tore Privé,
- des cases,
- des mutations directionnelles,
- de la cohérence fractale,
- de la reconstruction du fichier.

5.2. Taille des IDs : 16 bits

Tous les IDs du protocole sont sur **16 bits** :

- ID de case,
- ID du tore,
- ID transmis par le modèle 36 bits,
- ID reconstruit par le modèle 128 bits,
- ID muté localement (modèle 36 bits),
- ID muté structurellement (modèle 128 bits).

✓ Valeurs possibles

0 → 65535

✓ 100 % des valeurs sont utilisées

Aucune valeur n'est réservée. Aucune valeur n'est interdite. Aucune valeur n'est mise de côté.

5.3. Répétition des IDs

Les IDs peuvent se répéter **librement**, sans aucune limite.

Plusieurs cases peuvent partager le même ID, y compris :

- des dizaines,
- des centaines,
- des milliers,
- ou des dizaines de milliers de cases.

✓ Raison structurelle

- L'ID n'est pas un identifiant unique.
- L'ID est un **marqueur directionnel**.
- La sécurité repose sur la **géométrie directionnelle**, et non sur l'unicité des IDs.

✓ Conséquence

- 1 000 000 cases,
- 65 536 IDs possibles,
- répétitions normales,
- collisions normales,
- aucun impact sur la cohérence fractale.

5.4. Mutation des IDs

Deux types de mutation existent, selon le modèle directionnel choisi.

5.4.1. Mutation locale (modèle 36 bits)

(mise à jour TorUsNet 7.0)

Dans le modèle 36 bits :

- la clé fournit uniquement :
 - la case de départ,
 - l'ID de départ ;
- la clé ne parcourt pas le tore ;

- la clé n'exécute aucun déplacement ;
- la mutation locale est effectuée **par l'exécution du parcours directionnel interne**,
- aucune autre case n'est modifiée,
- aucune inversion normal/fantôme n'est effectuée.

Mutation locale :

$$ID_{\text{case}} \leftarrow ID_{\text{resultat du parcours}}$$

La mutation est donc **une conséquence du déplacement directionnel**, et non une action directe de la clé.

5.4.2. Mutation structurelle (modèle 128 bits)

Dans le modèle 128 bits :

- la clé interne parcourt le tore,
- inverse normal/fantôme,
- reconstruit le fichier,
- mute l'ID global du tore,
- mute l'ID de la case de départ,
- modifie la cohérence fractale.

Mutation structurelle :

$$ID_{\text{tore}} \leftarrow ID_{\text{aleatoire}}$$

$$ID_{\text{case départ}} \leftarrow ID_{\text{aleatoire}}$$

Les IDs aléatoires sont choisis sur 16 bits, même s'ils sont déjà utilisés.

5.5. Mutation aléatoire sur 16 bits

Lors d'une mutation, le programme choisit un ID **au hasard sur 16 bits**, même si cet ID est déjà utilisé.

✓ Propriétés

- mutation = tirage aléatoire sur 0–65535,
- aucune contrainte d'unicité,

- aucune contrainte de collision,
- aucune contrainte de disponibilité,
- mutation purement directionnelle.

5.6. Rôle des IDs dans le Tore Privé

Chaque case du tore possède :

- un ID sur 16 bits,
- un bit normal,
- un bit fantôme implicite,
- quatre directions internes,
- un déplacement torique,
- un saut exponentiel circulaire.

L'ID :

- identifie la case dans le fichier directionnel,
- sert de point d'entrée pour les mutations,
- sert de marqueur pour la cohérence fractale,
- sert de repère pour la reconstruction interne.

5.7. ID du tore

Le tore possède un **ID global sur 16 bits**.

Cet ID :

- change à chaque clé interne (modèle 128 bits),
- est choisi aléatoirement,
- peut se répéter,
- n'a aucune contrainte d'unicité.

Le modèle 36 bits ne modifie jamais l'ID global du tore.

5.8. Interaction entre IDs et modèles directionnels

✓ Modèle 36 bits

- fournit la case de départ,
- fournit l'ID de départ,
- déclenche un parcours interne,
- la mutation locale est effectuée par le parcours,
- ne modifie jamais l'ID du tore,
- n'inverse jamais normal/fantôme.

✓ **Modèle 128 bits**

- parcourt le tore,
- inverse normal/fantôme,
- reconstruit le fichier,
- mute l'ID du tore,
- mute l'ID de la case de départ,
- stabilise la cohérence fractale.

5.9. Cohérence fractale et IDs

La cohérence fractale :

- accepte les IDs répétés,
- accepte les collisions,
- accepte les mutations aléatoires,
- se stabilise automatiquement,
- se met à jour à chaque mutation directionnelle.

Les IDs ne sont pas une structure de sécurité. Ils sont une **structure directionnelle**.

5.10. Résumé du Chapitre 5

Le système des IDs fonctionne ainsi :

- tous les IDs sont sur 16 bits,
- 100 % des valeurs sont utilisées,
- les IDs peuvent se répéter librement,

- les mutations choisissent un ID aléatoire,
- le modèle 36 bits fournit la case de départ et mute via le parcours interne,
- le modèle 128 bits mute le tore et la case de départ,
- les collisions d'ID sont normales,
- la cohérence fractale reste stable,
- la sécurité repose sur la géométrie directionnelle, et non sur l'unicité des IDs.

Chapitre 6 — Handshake Directionnel (Séquence Host ↔ Client)

(Version TorUsNet 7.0 — mise à jour complète)

6.1. Vue d'ensemble

Le handshake directionnel est le mécanisme structurel qui établit une session cohérente entre un Host et un Client.

Contrairement aux protocoles classiques (TLS, SSH, IPSec) reposant sur :

- clés privées,
- certificats,
- signatures,
- dérivations mathématiques,
- échanges cryptographiques,

le handshake directionnel repose exclusivement sur :

- la géométrie interne des tores,
- la cohérence fractale,
- les déplacements directionnels,
- les IDs internes,
- la validation conjointe Host + Forteresse de Verre.

Il ne contient aucune cryptographie, aucune dérivation, aucune structure mathématique exploitable.

6.2. Objectifs du handshake

Le handshake directionnel assure :

1. la validation de la première clé directionnelle (36 bits ou 128 bits),
2. la synchronisation initiale entre Host et Client,
3. l'inscription de l'IP du Client comme IP de confiance,
4. la création de l'ID global de session (16 bits),
5. l'activation de la Forteresse de Verre,
6. l'initialisation du Tore Privé,
7. la mise en place de l'ID global du tore (16 bits),
8. la synchronisation directionnelle complète via **5 transmissions**.

Ces étapes sont structurelles, non cryptographiques.

6.3. La séquence officielle : 5 transmissions

La séquence directionnelle comporte **5 transmissions**, chacune ayant un rôle structurel précis. Elle est obligatoire, inviolable, et constitue la base de la synchronisation directionnelle.

Transmission 1 — Host → Client

Envoi initial de la clé directionnelle

Le Host envoie une clé directionnelle :

- **Modèle 36 bits :**
 - 20 bits : case de départ,
 - 16 bits : ID de départ.
- **Modèle 128 bits :**
 - 128 bits : clé directionnelle complète.

Le Client :

- reçoit la clé,
- vérifie sa structure,
- vérifie son modèle (36 ou 128 bits),
- vérifie l'ID visé,

- vérifie la cohérence fractale initiale,
- transmet la clé à la Forteresse de Verre pour validation conjointe.

Aucune exécution directionnelle n'est encore effectuée.

Transmission 2 — Client → Host

Renvoi de la clé pour confirmer la réception

Le Client renvoie **exactement la même clé** au Host.

Le Host :

- compare la clé envoyée et la clé reçue,
- valide la réception,
- inscrit l'IP du Client comme IP de confiance,
- active la Forteresse de Verre.

Si la clé diffère → **destruction immédiate**.

Transmission 3 — Host → Client

Autorisation d'exécution

Le Host renvoie une seconde copie de la clé.

Le Client :

- compare la clé renvoyée avec celle reçue,
- valide la cohérence,
- exécute le parcours directionnel interne dans son Tore Privé,
- met à jour les chemins internes,
- met à jour la cohérence fractale.

Effets selon le modèle :

✓ Modèle 36 bits

- mutation locale de l'ID de la case de départ,
- mutation effectuée **par le parcours**,
- aucune inversion normal/fantôme.

✓ Modèle 128 bits

- inversion normal/fantôme,
- mutation structurelle de l'ID du tore,
- mutation structurelle de l'ID de la case de départ.

Le Tore Privé du Client devient :

- vivant,
- oscillant,
- modifié.

Transmission 4 — Client → Host

Confirmation d'exécution du parcours

Le Client renvoie la clé après avoir exécuté le parcours interne.

Ce renvoi indique au Host :

- que la mutation locale (36 bits) ou structurelle (128 bits) a été effectuée,
- que les chemins internes ont été mis à jour,
- que la progression fractale est cohérente,
- que le Tore Privé du Client est à jour.

Le Host peut alors :

- exécuter le même parcours,
- appliquer les mêmes mutations,
- synchroniser son Tore Privé avec celui du Client.

Transmission 5 — Host → Client

Confirmation finale de synchronisation

Le Host renvoie la clé une dernière fois pour confirmer :

- qu'il a exécuté le parcours,
- qu'il a appliqué les mutations,
- que son Tore Privé est synchronisé avec celui du Client,

- que la session est directionnellement stable.

Le Client reçoit cette confirmation et la session devient :

- cohérente,
- stable,
- inviolable.

6.4. Pourquoi cinq transmissions ?

6.4.1. Validation bidirectionnelle complète

Host valide Client. Client valide Host. Puis chacun valide l'exécution directionnelle de l'autre.

6.4.2. Protection contre les injections

Un attaquant ne peut pas :

- intercepter la clé,
- la modifier,
- la renvoyer,
- la falsifier.

Toute incohérence est détectée structurellement.

6.4.3. Protection contre les répétitions

Une clé répétée :

- est détectée,
- est rejetée,
- détruit la session.

6.4.4. Protection contre les dérivations

Un attaquant ne peut pas dériver :

- la case de départ,
- l'ID de départ,
- les déplacements internes,

- les mutations,
- la cohérence fractale.

La structure directionnelle est non dérivable.

6.4.5. Synchronisation directionnelle parfaite

Les 5 transmissions garantissent que :

- les deux tores privés ont exécuté le même parcours,
- les mutations sont identiques,
- la géométrie interne est cohérente,
- aucune divergence n'est possible.

6.5. Synchronisation initiale

Une fois la clé validée :

- le Tore d'Authentification est utilisé une seule fois,
- le Tore Privé est initialisé,
- l'ID global est créé,
- les paires normal/fantôme sont définies (modèle 128 bits),
- les chemins internes sont activés,
- l'ID global du tore est établi,
- la Forteresse de Verre est armée.

Cette synchronisation est fractale, non mathématique.

6.6. Reconstruction initiale du Tore Privé

La première clé déclenche :

- l'initialisation directionnelle,
- la mise en place des chemins internes,
- la mutation locale (36 bits) ou structurelle (128 bits),
- la stabilisation fractale,
- l'activation des directions internes,

- l'activation des sauts exponentiels.

Le Tore Privé devient :

- vivant,
- oscillant,
- opérationnel.

6.7. Gestion des erreurs pendant le handshake

6.7.1. Clé incohérente

Si la clé :

- contient un ID invalide,
- contient une case impossible,
- contient un modèle illégitime,

→ la session est détruite.

6.7.2. Clé modifiée

Si la clé renvoyée diffère :

- destruction immédiate,
- effacement du Tore Privé,
- invalidation de l'IP.

6.7.3. Absence de réponse

Si le Host ne reçoit aucune réponse après 5 secondes :

- la clé est considérée perdue,
- une nouvelle clé est générée,
- avec une case de départ différente,
- mais reconstruisant la même structure directionnelle.

Aucune retransmission du même paquet n'est effectuée.

6.8. Résumé du Chapitre 6

Le handshake directionnel :

- repose sur **5 transmissions**,
- valide la première clé directionnelle,
- inscrit l'IP du Client comme IP de confiance,
- initialise le Tore Privé,
- active la Forteresse de Verre,
- crée l'ID global,
- établit l'ID unique du tore,
- garantit une cohérence directionnelle totale,
- protège contre les injections, répétitions, dérivations et falsifications.

Il constitue la base de la sécurité structurelle directionnelle.

Chapitre 7 — Multi Tore et Réseaux Parallèles

(Version TorUsNet 7.0 — mise à jour complète)

7.1. Vue d'ensemble

Le multi tore est l'un des fondements les plus puissants de l'architecture directionnelle. Il permet de créer plusieurs réseaux directionnels indépendants, chacun possédant :

- son propre **Tore Privé**,
- son propre **Tore d'Authentification**,
- ses propres **clés directionnelles** (36 bits ou 128 bits),
- ses propres **IDs internes**,
- son propre **ID global de session**,
- son propre **ID global du tore**,
- sa propre **Forteresse de Verre**.

Le **Tore Public**, pré-généré et universel, constitue la base directionnelle commune à toutes les branches.

Chaque réseau parallèle est totalement isolé des autres, même s'ils coexistent :

- sur la même machine,

- sur le même système,
- dans le même environnement réseau.

Le multi tore permet :

- la segmentation directionnelle,
- la souveraineté structurelle,
- l'isolation totale des branches,
- la création de réseaux privés inviolables,
- la duplication directionnelle contrôlée.

7.2. Définition d'un réseau parallèle

Un réseau parallèle est une instance complète de l'architecture directionnelle comprenant :

1. Le Tore Public (pré-généré, universel)
2. Un Tore d'Authentification
3. Un Tore Privé
4. Un ensemble de clés directionnelles
5. Une Forteresse de Verre
6. Un ID global de session
7. Un ID global du tore

Chaque réseau parallèle est un **univers directionnel indépendant**.

Il ne partage :

- aucune clé,
- aucun ID,
- aucun tore privé,
- aucune cohérence fractale,
- aucun chemin interne.

7.3. Création d'une nouvelle branche directionnelle

La création d'une nouvelle branche suit trois étapes.

7.3.1. Utilisation du Tore Public pré-généré

Le Tore Public :

- est pré-généré,
- est fourni avec le logiciel,
- est universel,
- est identique pour toutes les installations,
- est non modifiable,
- est non dérivable.

Il constitue la base directionnelle commune, mais **ne contient aucune information privée**.

7.3.2. Génération du Tore d'Authentification

Un Tore d'Authentification est généré. Il valide la première clé directionnelle et initialise :

- les premières paires normal/fantôme (modèle 128 bits),
- les premiers chemins internes,
- l'ID global de session,
- l'ID global du tore.

Ce tore n'est utilisé qu'une seule fois : lors du handshake initial.

7.3.3. Génération du Tore Privé

Un Tore Privé est généré. Il sera reconstruit fractalement à partir des clés directionnelles :

- inversion normal/fantôme (modèle 128 bits),
- mutation d'ID (locale ou structurelle),
- activation des chemins internes,
- stabilisation fractale.

Chaque Tore Privé est **indépendant** des autres tores privés existants.

7.4. Branches directionnelles

Chaque réseau parallèle est appelé **branche directionnelle**.

Une branche possède :

- son propre Tore Privé,
- sa propre progression fractale,
- ses propres IDs internes,
- sa propre Forteresse de Verre,
- son propre ID global,
- son propre ID global du tore.

7.4.1. Branches indépendantes

Les branches ne communiquent pas entre elles. Elles ne peuvent pas :

- partager des clés,
- partager des IDs,
- partager des tores privés,
- partager des chemins internes,
- partager des mutations.

7.4.2. Branches souveraines

Chaque branche est souveraine :

- elle ne dépend d'aucune autre branche,
- elle ne peut pas être influencée par une autre branche,
- elle ne peut pas être dérivée par une autre branche.

7.5. Duplication directionnelle (Tore primaire hors ligne)

Une fonctionnalité essentielle du multi tore est la **duplication directionnelle hors ligne**.

7.5.1. Tore primaire

Un tore primaire peut être :

- généré,
- stocké hors ligne,

- transféré physiquement,
- utilisé pour créer une nouvelle branche.

7.5.2. Transfert hors ligne

Le tore primaire peut être transféré :

- sur clé USB,
- sur disque dur,
- sur support physique,
- sur tout média non réseau.

Ce transfert ne contient **aucune clé directionnelle**. Il contient uniquement :

- la géométrie directionnelle,
- les IDs internes,
- les directions internes,
- les sauts exponentiels.

7.5.3. Création d'une branche à partir du tore primaire

Une fois le tore primaire transféré :

- une nouvelle branche est créée,
- un nouveau Tore Privé est reconstruit,
- un nouvel ID global est généré,
- un nouvel ID global du tore est établi,
- une nouvelle Forteresse de Verre est activée.

La branche est totalement indépendante de la branche d'origine.

7.6. Isolation structurelle

L'isolation structurelle est totale.

7.6.1. Isolation fractale

Les actions fractales d'une branche :

- ne peuvent pas influencer une autre branche,

- ne peuvent pas dériver une autre branche,
- ne peuvent pas reconstruire une autre branche.

7.6.2. Isolation directionnelle

Les chemins internes d'une branche :

- ne peuvent pas être utilisés dans une autre branche,
- ne peuvent pas être dérivés par une autre branche.

7.6.3. Isolation des IDs

Les IDs internes :

- ne sont pas compatibles entre branches,
- ne peuvent pas être transférés,
- ne peuvent pas être dérivés.

7.7. Multi Tore et sécurité post-quantique

Le multi tore renforce la sécurité post-quantique :

- un ordinateur quantique ne peut pas dériver un Tore Privé,
- ne peut pas dériver une branche,
- ne peut pas dériver une clé directionnelle,
- ne peut pas dériver un ID,
- ne peut pas dériver l'ID global du tore.

Même avec une puissance infinie, un attaquant ne peut pas :

- explorer un Tore Privé,
- dériver un Tore Privé,
- reconstruire un Tore Privé,
- analyser un Tore Privé.

Les tores privés sont **géométriquement inviolables**.

7.8. Multi Tore et sécurité photonique

Avec l'arrivée des ordinateurs photoniques :

- les structures mathématiques deviennent obsolètes,
- les cryptographies classiques deviennent vulnérables.

Le multi tore est conçu pour cette ère :

- les tores privés sont non transmissibles,
- non dérivables,
- non analysables,
- non explorables.

Un ordinateur photonique ne peut pas :

- pénétrer un Tore Privé,
- analyser un Tore Privé,
- dériver un Tore Privé.

7.9. Résumé du Chapitre 7

Le multi tore permet :

- la création de réseaux parallèles,
- l'isolation totale des branches,
- la duplication directionnelle hors ligne,
- la souveraineté structurelle,
- la sécurité post-quantique,
- la sécurité photonique.

Chaque branche est un univers directionnel indépendant, inviolable par conception. Le Tore Public, pré-généré et universel, sert de base directionnelle commune, mais **jamais de tore privé partagé**.

Chapitre 8 — Ports Directionnels et Géométrie Interne de Communication

(Version TorUsNet 7.0 — mise à jour complète)

8.1. Vue d'ensemble

Les ports directionnels de TorUsNet constituent une extension structurelle des ports classiques. Contrairement aux ports TCP/UDP (0–65535), TorUsNet utilise :

- des **ports étendus**,
- plusieurs **milliards de possibilités**,
- des ports **mutables**,
- **non scannables**,
- **non dérivables**,
- **non falsifiables**,
- validés structurellement par la **Forteresse de Verre**.

Un port directionnel n'est pas :

- un entier,
- un socket,
- un canal TCP,
- un identifiant réseau.

Un port directionnel est une **structure interne du Tore Privé**, générée directionnellement, mutée à chaque renouvellement du tore et à chaque nouvelle connexion.

8.2. Définition d'un port directionnel

Un port directionnel est :

- un **numéro de port étendu** (plusieurs milliards de possibilités),
- généré par le Tore Privé,
- muté à chaque renouvellement du tore,
- muté à chaque nouvelle connexion,
- impossible à prédire,
- impossible à scanner,
- impossible à dériver.

Règle structurelle : une seule connexion par port

Chaque port directionnel :

- n'autorise **qu'une seule connexion**,
- est invalidé dès que la session se termine,
- est remplacé par un nouveau port étendu lors de la prochaine connexion.

8.3. Pourquoi les ports directionnels sont inviolables

8.3.1. Absence dans la couche réseau classique

Un attaquant ne peut pas :

- scanner les ports,
- sonder les ports,
- ouvrir un port,
- envoyer un paquet vers un port.

Les ports directionnels **n'existent pas** dans la couche réseau classique. Ils n'existent que dans la mécanique interne du Tore Privé.

8.3.2. Ports mutables et non dérivables

Les ports directionnels :

- mutent automatiquement,
- ne sont jamais réutilisés,
- ne peuvent pas être prédits,
- ne peuvent pas être dérivés mathématiquement.

8.3.3. Validation par la Forteresse de Verre

La Forteresse de Verre :

- surveille les ports directionnels,
- valide leur cohérence,
- détruit la session si un port est violé,
- bloque l'IP hostile dans le pare-feu du système d'exploitation.

8.3.4. Ports non transmissibles

Un port directionnel :

- ne peut pas être envoyé,
- ne peut pas être copié,
- ne peut pas être exporté,
- ne peut pas être dérivé.

Il n'existe **que localement**, dans le Tore Privé.

8.4. Mutation des ports directionnels

8.4.1. Mutation lors du renouvellement du tore

Chaque fois que le Tore Privé est régénéré :

- un nouveau set de ports étendus est généré,
- tous les ports précédents deviennent invalides.

8.4.2. Mutation à chaque nouvelle connexion

À chaque nouvelle session :

- un nouveau port étendu est généré,
- une seule connexion est autorisée sur ce port,
- toute tentative de seconde connexion entraîne une **destruction immédiate**.

8.5. Ports directionnels et transmission

Host → Client

Le Host ouvre un port directionnel étendu. Le Client doit répondre sur ce même port.

Client → Host

Le Client renvoie la clé directionnelle sur ce port. Le Host valide la cohérence.

Host → Client

Le Host renvoie la clé finale sur le même port.

Violation d'un port

Si un port directionnel est violé :

- la session est détruite,
- le Tore Privé est effacé,

- l'IP est bloquée.

8.6. Ports directionnels et multi-tore

Dans un environnement multi-tore :

- chaque branche possède ses propres ports directionnels,
- les ports ne sont pas compatibles entre branches,
- les ports ne peuvent pas être dérivés d'une branche à l'autre.

Un port directionnel est **strictement local** à sa branche.

8.7. Ports directionnels et sécurité post-quantique

Les ports directionnels sont inviolables par un ordinateur quantique :

- impossible de dériver un port étendu,
- impossible de prédire une mutation,
- impossible d'explorer le Tore Privé,
- impossible de reconstruire les ports internes.

8.8. Ports directionnels et sécurité photonique

Les ordinateurs photoniques peuvent casser :

- RSA,
- ECC,
- TLS,
- SSH.

Mais ils ne peuvent pas :

- analyser une géométrie directionnelle,
- dériver un tore,
- reconstruire un port directionnel,
- prédire un port étendu muté.

Les ports directionnels sont **structurellement inviolables**.

8.9. Résumé du Chapitre 8

Les ports directionnels :

- sont des ports classiques étendus à plusieurs milliards de possibilités,
- mutent à chaque renouvellement du tore,
- mutent à chaque nouvelle connexion,
- n'autorisent qu'une seule connexion par port,
- sont non scannables,
- non dérivables,
- non falsifiables,
- validés par la Forteresse de Verre,
- inviolables par conception.

Ils constituent la **couche de communication interne** de TorUsNet, totalement indépendante des protocoles réseau classiques.

Chapitre 9 — Authentification DNS et HTTPS (Couche de Confiance Temporaire)

(Version TorUsNet 7.0 — mise à jour complète)

9.1. Vue d'ensemble

Avant que la géométrie directionnelle ne prenne le contrôle total de la session, une **couche de confiance temporaire** est nécessaire pour :

- identifier l'hôte demandé,
- éviter les redirections malveillantes,
- empêcher les attaques MITM classiques,
- sécuriser la transmission de la première clé directionnelle,
- garantir que le Client contacte bien le bon Host.

Cette couche repose sur deux technologies de l'Internet classique :

- **DNS** (résolution du nom de domaine),
- **HTTPS** (certificat TLS).

Ces technologies ne font pas partie de la mécanique directionnelle. Elles servent uniquement de **sas d'entrée**, avant l'activation :

- du Tore d'Authentification,
- du Tore Privé,
- de la Forteresse de Verre,
- des IDs internes,
- de l'ID global,
- de l'ID global du tore.

Une fois la première clé validée, **DNS et HTTPS disparaissent complètement**.

9.2. Rôle du DNS dans TorUsNet

DNS est utilisé pour une seule raison :

Identifier l'hôte demandé avant d'entrer dans la géométrie directionnelle.

9.2.1. Vérification du domaine

Lorsqu'un Client contacte un Host :

1. DNS résout le nom de domaine.
2. Le protocole vérifie que ce domaine appartient à la liste des hôtes de confiance.
3. Si le domaine est inconnu ou suspect :
 - la connexion passe en mode confirmation utilisateur,
 - aucune clé directionnelle n'est transmise.

9.2.2. Protection contre les redirections

DNS protège contre :

- les redirections malveillantes,
- les faux domaines,
- les hôtes clonés,
- les détournements de trafic.

9.2.3. Rôle strictement temporaire

Une fois la première clé validée :

DNS n'est plus utilisé. La géométrie directionnelle prend le relais.

9.3. Rôle du HTTPS dans TorusNet

HTTPS est utilisé pour :

Sécuriser la transmission de la première clé directionnelle.

9.3.1. Vérification du certificat

Le Client vérifie :

- le certificat TLS,
- la validité du domaine,
- l'absence d'interception MITM classique.

Si le site n'a pas de HTTPS :

- la connexion passe en mode confirmation utilisateur,
- aucune clé directionnelle n'est envoyée.

9.3.2. Protection contre les attaques classiques

HTTPS protège contre :

- les interceptions,
- les injections,
- les manipulations de trafic,
- les attaques de downgrade.

9.3.3. Rôle strictement temporaire

Une fois la première clé transmise :

HTTPS disparaît. La session devient 100 % directionnelle.

9.4. Interaction DNS ↔ HTTPS ↔ Tore d'Authentification

La séquence complète est :

1. DNS identifie l'hôte.
2. HTTPS sécurise la première transmission.

3. Le Tore d'Authentification valide la première clé.
4. La Forteresse de Verre active la session directionnelle.
5. Le Tore Privé prend le contrôle.
6. DNS et HTTPS sont abandonnés.

9.4.1. Cas de confiance totale

- DNS valide
- HTTPS valide

→ La première clé est transmise automatiquement.

9.4.2. Cas de confiance partielle

- DNS valide
- HTTPS absent

→ Confirmation utilisateur obligatoire.

9.4.3. Cas de suspicion

- DNS suspect
- HTTPS absent ou invalide

→ Connexion bloquée jusqu'à validation explicite.

9.5. Pourquoi TorUsNet utilise encore DNS et HTTPS

TorUsNet est une architecture directionnelle post-Internet. Cependant, l'écosystème actuel impose :

- l'usage de noms de domaine,
- l'usage de certificats TLS,
- l'usage de serveurs HTTP(S).

DNS et HTTPS servent donc de **pont** entre :

- l'Internet classique,
- l'Internet directionnel.

9.5.1. Objectif final : disparition totale

Dans les versions futures :

- DNS sera remplacé par un **DNS directionnel**,
- HTTPS sera remplacé par une **validation directionnelle native**,
- les certificats seront remplacés par des **IDs directionnels**,
- les connexions seront entièrement **géométriques**.

9.6. Interaction avec les mises à jour directionnelles

Inversion et mutation d'ID

Une fois la première clé validée :

- l'ID de la case de départ mute,
- l'ID global est créé,
- DNS et HTTPS ne sont plus nécessaires.

Session directionnelle autonome

La session devient :

- continue,
- vivante,
- autonome.

9.7. Résumé du Chapitre 9

Le chapitre établit que :

- DNS et HTTPS ne servent qu'à sécuriser la première clé,
- ils disparaissent immédiatement après validation,
- la session devient 100 % directionnelle,
- la Forteresse de Verre prend le contrôle,
- le Tore Privé devient l'unique couche de communication,
- les IDs internes remplacent les certificats,
- les ports directionnels remplacent les sockets,
- la géométrie directionnelle remplace la cryptographie.

Mise à Jour — Abandon de l'authentification par adresse MAC, ID processeur et carte mère

(Version TorUsNet 7.0 — Mise à jour structurelle)

Contexte de la modification

Les premières versions expérimentales de TorUsNet utilisaient une authentification matérielle basée sur :

- l'adresse MAC,
- l'ID du processeur,
- l'ID de la carte mère.

Cette approche avait été envisagée pour renforcer la validation initiale du Host et du Client avant l'activation du Tore d'Authentification.

Cependant, cette méthode a été **abandonnée dans TorUsNet 7.0** pour des raisons structurelles, techniques et légales.

1. Raison principale : caractère confidentiel des données matérielles

Les identifiants matériels (MAC, CPU ID, Motherboard ID) sont considérés comme :

- des données sensibles,
- des informations personnelles,
- des éléments confidentiels,
- des marqueurs persistants non conformes à la philosophie de Tor et de TorUsNet.

TorUsNet ne peut pas dépendre d'informations privées.

L'architecture directionnelle repose sur :

- la géométrie interne,
- les tores privés,
- les IDs directionnels,
- la cohérence fractale,
- la Forteresse de Verre.

Elle ne doit jamais reposer sur des identifiants matériels persistants.

2. Conformité avec les principes fondamentaux de Tor

Le réseau Tor impose une règle essentielle :

Aucune donnée matérielle ne doit être utilisée pour identifier un utilisateur.

TorUsNet, en tant qu'architecture directionnelle post-Tor, doit respecter cette base :

- pas d'adresse MAC,
- pas d'ID processeur,
- pas d'ID carte mère,
- pas d'identifiant matériel persistant.

Ces données sont incompatibles avec :

- l'anonymat structurel,
- la non-traçabilité,
- la non-corrélation,
- la souveraineté directionnelle.

3. Raison technique : incompatibilité avec la géométrie directionnelle

Les identifiants matériels :

- ne sont pas directionnels,
- ne sont pas fractals,
- ne sont pas mutables,
- ne peuvent pas être intégrés dans un tore,
- ne peuvent pas être validés par la Forteresse de Verre.

Ils introduisent une **structure statique**, alors que TorUsNet repose sur :

- des IDs mutables,
- des ports directionnels mutables,
- des tores vivants,
- des parcours internes,
- des mutations fractales.

Un identifiant matériel ne peut pas suivre la logique directionnelle.

4. Raison de sécurité : impossibilité de garantir l'intégrité matérielle

Les identifiants matériels peuvent être :

- falsifiés,
- clonés,
- virtualisés,
- masqués,
- modifiés par le système d'exploitation.

Ils ne constituent pas une base fiable pour une architecture directionnelle.

TorUsNet 7.0 exige une validation :

- interne,
- géométrique,
- fractale,
- non transmissible,
- non dérivable.

Les identifiants matériels ne répondent à aucune de ces exigences.

5. Remplacement complet par la validation directionnelle

Dans TorUsNet 7.0, l'authentification matérielle est remplacée par :

✓ Le Tore d'Authentification

Valide la première clé directionnelle.

✓ La Forteresse de Verre

Valide la cohérence structurelle du Host.

✓ Les IDs directionnels

Mutable, non dérivables, non transmissibles.

✓ Les ports directionnels

Mutable, non scannables, non falsifiables.

✓ Le handshake directionnel en 5 transmissions

Synchronisation complète Host ↔ Client.

Aucune donnée matérielle n'est utilisée.

6. Conclusion de la mise à jour

L'authentification via :

- adresse MAC,
- ID processeur,
- ID carte mère,

a été **définitivement abandonnée** dans TorUsNet 7.0.

Cette décision garantit :

- la conformité avec les principes de Tor,
- la confidentialité structurelle,
- la non-traçabilité,
- la souveraineté directionnelle,
- la cohérence fractale,
- la sécurité post-quantique et photonique.

TorUsNet 7.0 repose désormais **exclusivement** sur la géométrie directionnelle et les mécanismes internes du tore.